In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score, confusion_matrix

# ---------- 1. Load and clean ----------
# Own dataset irundha: CSV_PATH = r"C:\Users\srini\Downloads\credit data.csv" nu maathunga
CSV_PATH = None
TARGET = &#x27;default'   # 1 = defaulted (bad customer), 0 = repaid (good customer)

if CSV_PATH:
    df = pd.read_csv(CSV_PATH)
else:
    rng = np.random.default_rng(42)
    n = 3000
    df = pd.DataFrame({
        &#x27;age': rng.integers(21, 65, n),
        &#x27;income': rng.normal(50000, 18000, n).clip(12000),
        &#x27;loan_amount': rng.normal(15000, 8000, n).clip(1000),
        &#x27;credit_score': rng.normal(680, 70, n).clip(300, 850),
        &#x27;employment_years': rng.integers(0, 30, n),
        &#x27;late_payments': rng.poisson(1.2, n),
        &#x27;credit_utilization': rng.uniform(0, 1, n),
        &#x27;home_owner': rng.integers(0, 2, n),
    })
    df[&#x27;dti'] = df['loan_amount'] / df['income']
    z = (-1.0 + 3.0 * df[&#x27;dti'] + 0.6 * df['late_payments']
         + 1.5 * df[&#x27;credit_utilization']
         - 0.012 * (df[&#x27;credit_score'] - 650)
         - 0.03 * df[&#x27;employment_years'] - 0.4 * df['home_owner'])
    p = 1 / (1 + np.exp(-z))
    df[TARGET] = (rng.uniform(size=n) < p).astype(int)

df = df.dropna().drop_duplicates().reset_index(drop=True)
print(df.head())
print("Shape:", df.shape)
print("Default rate:", round(df[TARGET].mean() * 100, 2), "%")

# ---------- 2. Univariate analysis ----------
def univariate_analysis(df, column):
    freq = df[column].value_counts()
    mean = df[column].mean()
    median = df[column].median()
    mode = df[column].mode()[0]
    std = df[column].std()
    return freq, mean, median, mode, std

freq, mean, median, mode, std = univariate_analysis(df, &#x27;credit_score')
print("\nUnivariate Analysis for Credit Score")
print("Frequency (top 5):\n", freq.head())
print("Mean:", mean)
print("Median:", median)
print("Mode:", mode)
print("Standard Deviation:", std)

plt.figure(figsize=(10, 4))
plt.hist(df[&#x27;credit_score'], bins=30)
plt.title("Credit Score Distribution")
plt.xlabel("Credit Score")
plt.ylabel("Number of Customers")
plt.show()

# ---------- 3. Factors influencing credit risk ----------
features = [c for c in df.columns if c != TARGET]

corr = df.corr(numeric_only=True)[TARGET].drop(TARGET).sort_values()
print("\nCorrelation of each factor with Default")
print(corr)

plt.figure(figsize=(10, 4))
plt.barh(corr.index, corr.values)
plt.title("Factors vs Default (Correlation)")
plt.xlabel("Correlation with Default")
plt.show()

# Good vs Bad customers - average of each factor
print("\nAverage factor values: Good (0) vs Bad (1)")
print(df.groupby(TARGET)[features].mean().T)

# ---------- 4. Train-test split ----------
X = df[features]
y = df[TARGET]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
print("\nTotal, Train, Test:", len(df), len(X_train), len(X_test))

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

```
# ---------- 5. Simple logistic regression ----------
simple_model = LogisticRegression()
simple_model.fit(X_train[[&#x27;credit_score']], y_train)
pred_simple = simple_model.predict(X_test[[&#x27;credit_score']])
prob_simple = simple_model.predict_proba(X_test[[&#x27;credit_score']])[:, 1]
acc_simple = accuracy_score(y_test, pred_simple)
auc_simple = roc_auc_score(y_test, prob_simple)

print("\nSimple Logistic Regression (Credit Score only)")
print("Coefficient:", simple_model.coef_[0][0])
print("Intercept:", simple_model.intercept_[0])
print("Accuracy:", acc_simple)
print("ROC-AUC:", auc_simple)

# ---------- 6. Multiple logistic regression ----------
multi_model = LogisticRegression(max_iter=1000)
multi_model.fit(X_train_s, y_train)
pred_multi = multi_model.predict(X_test_s)
prob_multi = multi_model.predict_proba(X_test_s)[:, 1]
acc_multi = accuracy_score(y_test, pred_multi)
auc_multi = roc_auc_score(y_test, prob_multi)

print("\nMultiple Logistic Regression (all factors)")
coef = pd.Series(multi_model.coef_[0], index=features).sort_values()
print("Coefficients (+ increases risk, - decreases risk):\n", coef)
print("Intercept:", multi_model.intercept_[0])
print("Accuracy:", acc_multi)
print("ROC-AUC:", auc_multi)

plt.figure(figsize=(10, 4))
plt.barh(coef.index, coef.values)
plt.title("Factor Influence on Default Risk")
plt.xlabel("Coefficient")
plt.show()

# ---------- 7. Naive baseline ----------
majority = y_train.mode()[0]
acc_naive = accuracy_score(y_test, [majority] * len(y_test))
print("\nNaive baseline Accuracy (everyone = majority class):", acc_naive)

# ---------- 8. Comparison ----------
print("\nModel Comparison")
print("Simple Model Accuracy   :", acc_simple, "| AUC:", auc_simple)
print("Multiple Model Accuracy :", acc_multi, "| AUC:", auc_multi)
print("Naive Baseline Accuracy :", acc_naive)
print("Better model:", "Simple" if auc_simple > auc_multi else "Multiple")

print("\nConfusion Matrix (Multiple Model):\n", confusion_matrix(y_test, pred_multi))

# ---------- 9. Classify customers into risk classes ----------
def risk_class(p):
    if p < 0.30:
        return "Low Risk"
    elif p < 0.60:
        return "Medium Risk"
    return "High Risk"

result = X_test.copy()
result[&#x27;default_probability'] = prob_multi
result[&#x27;risk_class'] = result['default_probability'].apply(risk_class)
result[&#x27;actual_default'] = y_test.values

print("\nCustomers per Risk Class")
print(result[&#x27;risk_class'].value_counts())
print("\nActual default rate per Risk Class")
print(result.groupby(&#x27;risk_class')['actual_default'].mean())

order = ["Low Risk", "Medium Risk", "High Risk"]
counts = result[&#x27;risk_class'].value_counts().reindex(order).fillna(0)
plt.figure(figsize=(10, 4))
plt.bar(counts.index, counts.values)
plt.title("Customer Classification by Credit Risk")
plt.xlabel("Risk Class")
plt.ylabel("Number of Customers")
plt.show()

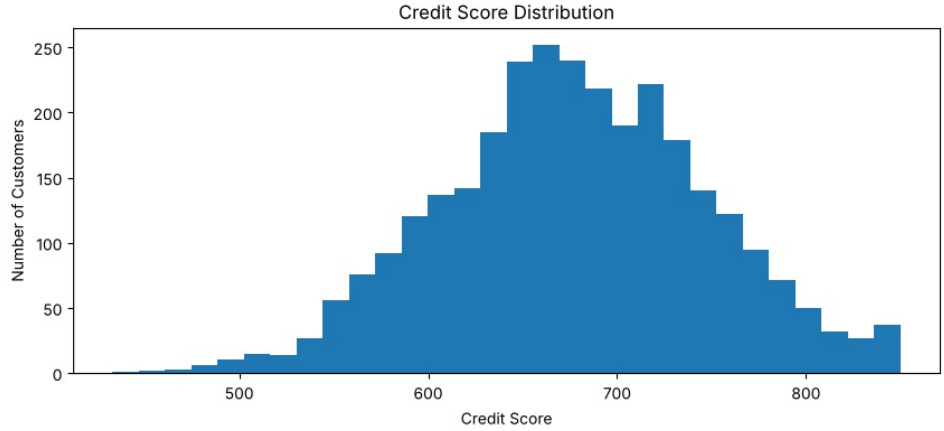
result.to_csv("customer_risk_classification.csv", index=False)
```

```
   age        income   loan_amount  ...  home_owner       dti  default
0   24  67333.290318  18053.524276  ...           1  0.268122        1
1   55  36957.001073  14573.001245  ...           1  0.394323        0
2   49  34632.333993  23690.607335  ...           1  0.684060        1
3   40  58274.707190  10064.163970  ...           1  0.172702        0
4   40  67793.958364  22889.392383  ...           0  0.337632        1

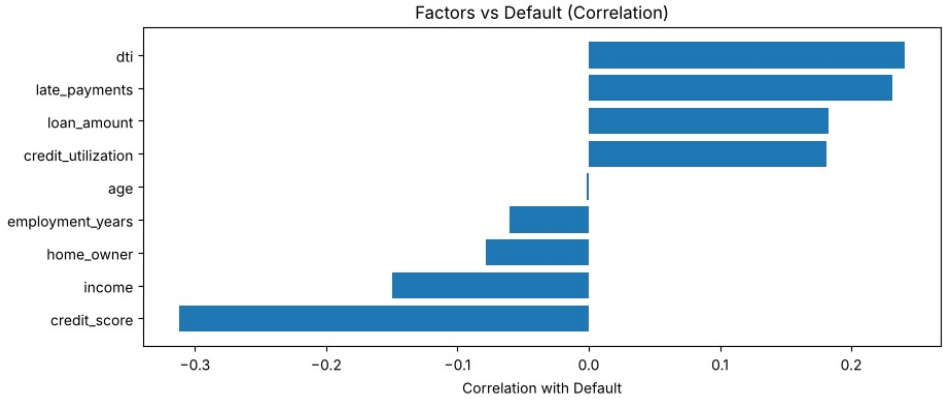
[5 rows x 10 columns]
Shape: (3000, 10)
Default rate: 60.47 %

Univariate Analysis for Credit Score
Frequency (top 5):
 credit_score
850.000000    22
704.563908     1
```

```
669.202722     1
753.448784     1
624.059932     1
Name: count, dtype: int64
Mean: 677.574438962082
Median: 676.7308926782696
Mode: 850.0
Standard Deviation: 70.1796892065653
```




```
Correlation of each factor with Default
credit_score         -0.311767
income               -0.149886
home_owner           -0.078627
employment_years     -0.060053
age                  -0.001855
credit_utilization    0.180842
loan_amount           0.182835
late_payments         0.231646
dti                   0.240631
Name: default, dtype: float64
```

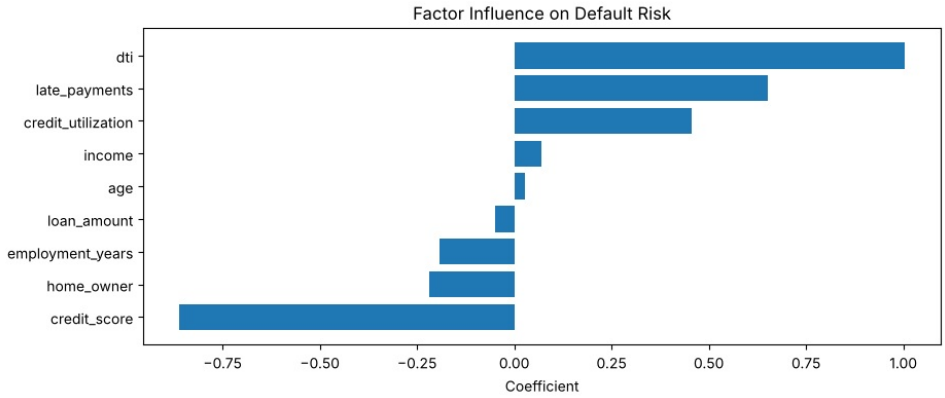



```
Average factor values: Good (0) vs Bad (1)
default                         0             1
age                     42.575885     42.527563
income               53071.404009  47627.567857
loan_amount          13496.503982  16394.928874
credit_score           704.629342    659.885842
employment_years        15.209106     14.145535
late_payments            0.910624      1.427784
credit_utilization       0.447132      0.554852
home_owner               0.537943      0.457552
dti                      0.279976      0.423683

Total, Train, Test: 3000 2400 600

Simple Logistic Regression (Credit Score only)
Coefficient: -0.009736849413649454
Intercept: 7.060956463098058
Accuracy: 0.6633333333333333
ROC-AUC: 0.7016540549336867

Multiple Logistic Regression (all factors)
Coefficients (+ increases risk, - decreases risk):
 credit_score         -0.860541
home_owner           -0.217970
employment_years     -0.191137
loan_amount          -0.049174
age                   0.028524
income                0.069318
credit_utilization    0.455165
late_payments         0.650541
dti                   1.003183
dtype: float64
Intercept: 0.6490902201383115
Accuracy: 0.7216666666666667
ROC-AUC: 0.7990840510978601
```




```
Naive baseline Accuracy (everyone = majority class): 0.605

Model Comparison
Simple Model Accuracy   : 0.6633333333333333 | AUC: 0.7016540549336867
Multiple Model Accuracy : 0.7216666666666667 | AUC: 0.7990840510978601
Naive Baseline Accuracy : 0.605
Better model: Multiple

Confusion Matrix (Multiple Model):
 [[149  88]
 [ 79 284]]

Customers per Risk Class
risk_class
High Risk      312
Medium Risk    195
Low Risk        93
Name: count, dtype: int64

Actual default rate per Risk Class
risk_class
High Risk      0.807692
Low Risk       0.204301
Medium Risk    0.471795
Name: actual_default, dtype: float64
```

